Analytics Engineering: Session 9

Advanced Testing, Documentation & Exposures

Agenda

- Recap: Generic Tests
- Singular Tests
- Custom Generic Tests
- Advanced Tests with dbt-expectations
- Documentation & Doc Blocks
- Exposures
- Generating the Docs Site

Recap: Generic Tests

We introduced `unique` and `not_null` in Session 3. The full set of built-in generic tests:

Test	What it checks
`unique`	No duplicate values
`not_null`	No null values
`accepted_values`	Values from a known set
`relationships`	Foreign key integrity

```yaml
- name: status
  tests:
    - accepted_values:
        values: ['pending', 'shipped', 'delivered', 'cancelled']
```

Singular Tests

Custom SQL queries in `tests/`. If the query returns rows, the test **fails**.

```
-- tests/assert_total_payment_positive.sql
select order_id, total_amount
from {{ ref('mart_orders') }}
where total_amount < 0
```

Use singular tests for business rules that don't fit generic tests.

Custom Generic Tests

Write your own **reusable** test macros in `macros/`. These work just like built-in generic tests.

```
-- macros/test_is_positive.sql
{% test is_positive(model, column_name) %}
    select *
    from {{ model }}
    where {{ column_name }} < 0
{% endtest %}
```

Usage in YAML (just like `unique` or `not_null`):

```
columns:
  - name: total_amount
    tests:
      - is_positive
```

Advanced Tests with dbt-expectations

The `dbt_expectations` package provides dozens of advanced data quality tests:

```yaml
columns:
  - name: email
    tests:
      - dbt_expectations.expect_column_values_to_match_regex:
          regex: "^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\\.[a-zA-Z0-9-.]+$"
  - name: order_date
    tests:
      - dbt_expectations.expect_column_values_to_be_between:
          min_value: "'2020-01-01'"
          max_value: "'2030-12-31'"
  - name: quantity
    tests:
      - dbt_expectations.expect_column_values_to_be_between:
          min_value: 1
```

Testing Strategy by Layer

Layer	Test Focus	Example Tests
Staging	Data integrity	`unique`, `not_null` on PKs
Intermediate	Join correctness	`relationships`, row count checks
Marts	Business rules	`accepted_values`, singular tests, custom generic

Documenting Models

Add descriptions to YAML files alongside your tests:

```yaml
models:
  - name: mart_orders
    description: "One row per order with item summary and shipping metrics."
    columns:
      - name: order_id
        description: "Primary key. Unique identifier for each order."
        tests:
          - unique
          - not_null
      - name: total_amount
        description: "Total order value in USD."
```

- **Coverage**: Aim for 100% documentation on Marts.

Doc Blocks

For long descriptions, use `{% docs %}` blocks in markdown files:

```
←!— models/marts/docs/mart_orders_description.md —→
{% docs mart_orders %}

This model contains one row per order, enriched with:
- **Item metrics**: total items, total quantity, total price
- **Shipping metrics**: days to ship, late delivery flag

Used by the Revenue Dashboard and the Operations team.

{% enddocs %}
```

Reference in YAML:

```yaml
- name: mart_orders
  description: '{{ doc("mart_orders") }}'
```

Exposures

Define downstream consumers (dashboards, ML models, reports) in YAML.

```yaml
exposures:
  - name: weekly_revenue_dashboard
    type: dashboard
    description: "Weekly revenue breakdown by segment and product category."
    owner:
      name: Data Team
      email: data@company.com
    depends_on:
      - ref('mart_orders')
      - ref('mart_revenue_by_segment')
```

Why Exposures Matter

- **Visibility**: Exposures appear in the lineage graph, showing what depends on your models.
- **Impact Analysis**: Before changing `mart_orders`, you can see which dashboards will be affected.
- **Ownership**: Document who owns each downstream consumer.
- **Certification**: Exposures are tested under Domain 7 (External Dependencies).

Generating the Docs Site

```
dbt docs generate    # Compile metadata into catalog.json
dbt docs serve       # Host a local website to browse docs
```

The docs site includes:

- **Model descriptions** and column details
- **Lineage graph** (interactive DAG visualization)
- **Test coverage** for each model
- **Exposures** shown as downstream nodes in the DAG

What have we learned in this session

- Write singular tests for custom business rules
- Create custom generic test macros for reusable validation
- Apply advanced tests with dbt-expectations
- Document models, columns, and sources in YAML
- Write doc blocks for complex descriptions
- Define exposures for downstream consumers
- Generate and explore the documentation site

Next Session: Advanced Materializations: Incremental & Snapshots.